	**Carátula para entrega de prácticas**

Laboratorio de computación salas A y B

Profesor: René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 7

Integrante(s): 120002850 323064039

323071378 323214636

426059576 426058744

Semestre: 2027-1

Fecha de entrega: 16-Octubre-2026

Obervaciones:

CALIFICACIÓN:

Índice

1. Introducción **3**
1.1. Planteamiento del problema 3
1.2. Motivación 3
1.3. Objetivos 3

2. Marco Teórico **4**

3. Desarrollo **5**
3.1. Investigación y análisis de consultas 5
3.2. Diseño de clases y jerarquía de herencia 11

4. Resultados **18**

5. Conclusiones **20**

1. Introducción

1.1. Planteamiento del problema

El modelado de entidades geométricas complejas en el desarrollo de software requiere estructuras organizadas que reflejen con precisión las propiedades matemáticas de las figuras y eviten la redundancia de datos. En particular, la familia de los cuadriláteros comparte una definición fundamental basada en cuatro vértices coplanares en el plano bidimensional, pero diverge significativamente en las restricciones impuestas sobre el paralelismo de sus lados, la congruencia de sus aristas y la perpendicularidad de sus ángulos internos. El desafío académico y técnico consiste en concebir e implementar en el lenguaje Java una jerarquía de herencia lo más profunda posible que articule las clases Cuadrilátero, Trapezoide, Paralelogramo, Rectángulo y Cuadrado, apoyándose en una clase Punto para la gestión de coordenadas, de manera que cada nivel refine progresivamente el cálculo de su área superficial sin alterar la superclase común.

1.2. Motivación

La comprensión profunda del mecanismo de herencia simple y la especialización progresiva de clases constituye uno de los pilares formativos centrales en la programación orientada a objetos. Estudiar la transición sistemática desde una abstracción geométrica general hacia formas altamente restringidas permite asimilar cómo el principio de sustitución de Liskov y el polimorfismo facilitan la extensibilidad y robustez del código fuente. Asimismo, abordar este problema mediante una jerarquía multinivel en lugar de clases aisladas fomenta hábitos rigurosos de encapsulamiento, promueve la reutilización eficiente de atributos y métodos, y demuestra cómo las fórmulas de la geometría analítica se traducen de forma natural en algoritmos modulares y verificables.

1.3. Objetivos

El propósito primordial de esta práctica radica en diseñar, programar y validar una jerarquía de herencia en Java con la máxima profundidad factible, estableciendo a la clase Cuadrilátero como la superclase raíz dotada de cuatro variables privadas de tipo Punto. Como metas específicas, se busca implementar las subclases Trapezoide, Paralelogramo, Rectángulo y Cuadrado vinculadas linealmente mediante herencia, derivar en cada una el método de cálculo de área correspondiente a su definición geométrica formal, omitir explícitamente dicho cálculo en la superclase Cuadrilátero por carecer de restricciones suficientes, y contrastar los resultados obtenidos en ejecución contra soluciones analíticas calculadas previamente sobre un conjunto estandarizado de pruebas y casos frontera.

2. Marco Teórico

La programación orientada a objetos se sustenta en la creación de abstracciones computacionales capaces de representar entidades del dominio del problema mediante atributos que definen su estado y métodos que encapsulan su comportamiento [1]. Dentro de este paradigma, el concepto de herencia permite definir nuevas clases a partir de clases preexistentes, estableciendo una relación taxonómica de tipo "es-un" (*is-a*) entre subclases y superclases. Cuando una subclase extiende a una superclase en Java mediante la palabra reservada `extends`, adquiere de forma automática la accesibilidad a sus miembros públicos y protegidos, permitiendo añadir nuevos atributos, declarar métodos especializados o sobrescribir implementaciones previas para adaptarlas a un propósito más específico [2].

En jerarquías de herencia profunda, la especialización ocurre a lo largo de múltiples niveles estratificados, donde cada clase derivada representa una instancia conceptualmente más restringida que su antecesora inmediata. Este principio concuerda de manera perfecta con la clasificación geométrica de los polígonos de cuatro lados: todo cuadrado es intrínsecamente un rectángulo con lados adyacentes congruentes; todo rectángulo es un paralelogramo con ángulos rectos; todo paralelogramo es un trapezoide con ambos pares de lados opuestos paralelos; y todo trapezoide es un cuadrilátero plano general. Diseñar una jerarquía que capture con precisión esta cadena evita la duplicación estructural de código y asegura que las propiedades geométricas invariantes se conserven intactas a lo largo de toda la estructura de herencia [3].

Desde la perspectiva del diseño de software, el encapsulamiento garantiza que el estado interno de los objetos se mantenga resguardado contra manipulaciones indebidas desde el exterior. Al definir las coordenadas de los vértices como variables de instancia privadas en la clase base y exponer su consulta únicamente a través de métodos de acceso (*getters*), se preserva la integridad de los datos geométricos. Además, la decisión de no incluir un método de cálculo de área en la clase base responde a un principio básico de coherencia arquitectónica: un cuadrilátero arbitrario en el plano no cuenta con una fórmula cerrada única basada exclusivamente en la multiplicación de bases y alturas sin recurrir a descomposiciones triangulares complejas, mientras que sus derivadas sí disponen de expresiones analíticas deterministas directamente ligadas a sus restricciones geométricas particulares [1].

3. Desarrollo

3.1. Investigación y análisis de consultas

Como eje metodológico de la investigación previa, el equipo formuló una consulta técnica uniforme orientada a dilucidar un aspecto arquitectónico crítico en el diseño de jerarquías orientadas a objetos: *¿Por qué el modificador de acceso* `protected` *en variables de instancia suele considerarse una mala práctica en la herencia?* Esta decisión de diseño resulta determinante para la implementación de la superclase `Cuadrilatero`, donde los cuatro vértices coordenados deben mantenerse como atributos estrictamente privados (`private`) con métodos accesores en lugar de exponerse directamente a las subclases descendientes.

En la figura 3.1 se documenta la consulta formulada por Alan mediante el modelo Google Gemini. El análisis resalta que declarar campos `protected` quebranta el principio de encapsulamiento y el modelo de caja negra", al permitir que las clases derivadas manipulen directamente el estado interno de la clase base, provocando un acoplamiento estrecho (*tight coupling*) que rigidiza el sistema ante refactorizaciones futuras.

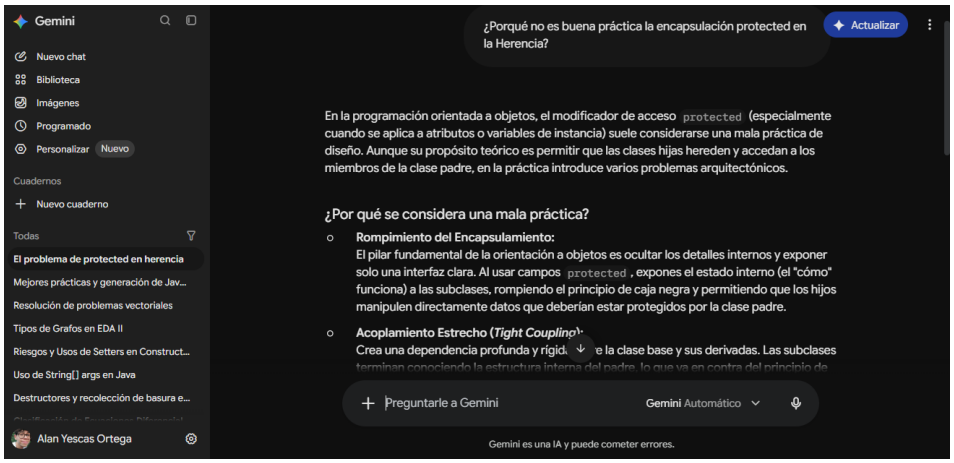


Figura 3.1: Consulta de investigación 1: Alan (Google Gemini)

Por su parte, en la figura 3.2 se presenta la respuesta obtenida por Zoé en ChatGPT, donde a través de una clasificación comparativa de modificadores de acceso en Java se enfatiza cómo `protected` debilita la modularidad al permitir visibilidad no solo a subclases, sino a cualquier otra clase dentro del mismo paquete, exponiendo datos sensibles a alteraciones imprevistas y vulnerando el principio de mínimo privilegio en el desarrollo orientado a objetos.

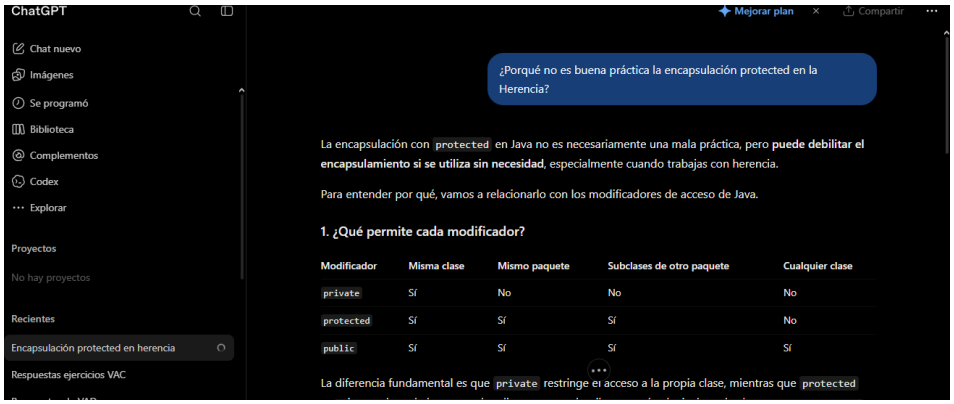


Figura 3.2: Consulta de investigación 2: Zoé (ChatGPT)

En la figura 3.3 se expone la captura de Luis, generada en un entorno móvil de ChatGPT. La respuesta puntualiza que el acceso directo desde clases hijas prescinde de las validaciones de consistencia que proporcionan los métodos accesores y mutadores (`getters` y `setters`), lo cual compromete la integridad estructural de la información geométrica y dificulta el mantenimiento a largo plazo. Asimismo, desaconseja rotundamente que las subclases modifiquen arbitrariamente las coordenadas sin pasar por la lógica de control de la superclase.

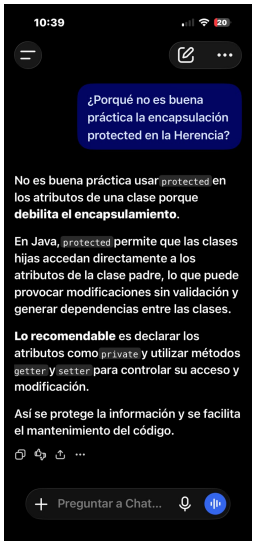


Figura 3.3: Consulta de investigación 3: Luis (ChatGPT Móvil)

Continuando con la evaluación de principios de diseño, en la figura 3.4 se exhibe la consulta sintética de Joseph utilizando el modelo MiMo-V2.6-Pro UltraSpeed. Su respuesta de alta densidad técnica advierte sobre el fenómeno de la Clase Base Frágil" (*Fragile Base Class problem*): cualquier cambio en la estructura de la clase padre puede romper inadvertidamente a las subclases dependientes. Joseph concluye que debe privilegiarse la herencia de contrato e interfaz controlada sobre la herencia de implementación interna.

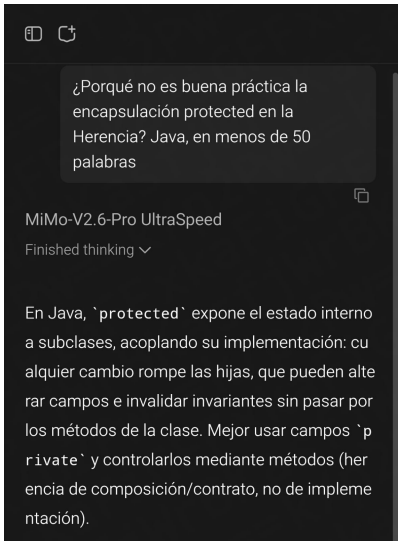


Figura 3.4: Consulta de investigación 4: Joseph (MiMo UltraSpeed)

En la figura 3.5 se observa la interacción aportada por Hanniel a través del asistente de IA. La consulta corrobora que la exposición de atributos protegidos deteriora la cohesión del sistema y eleva innecesariamente la superficie de error, recomendando como directriz prioritaria reservar los campos de estado como estrictamente privados y canalizar todas las interacciones a través de métodos accesores públicos explícitos.

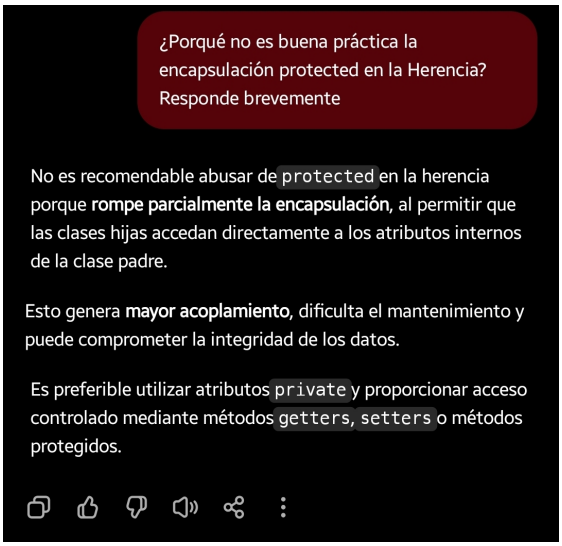


Figura 3.5: Consulta de investigación 5: Hanniel (Asistente IA)

Por su parte, en la figura 3.6 se aprecia el análisis estructurado de Alex, el cual desglosa el problema en tres principios formales de la ingeniería de software: ruptura de la encapsulación real e invalidación de invariantes de estado, acoplamiento fuerte y fragilidad de clases derivadas, y violación directa de los principios SOLID de Responsabilidad Única (SRP) y Abierto/Cerrado (OCP). La convergencia de estos seis análisis fundamentó la decisión unánime del equipo de declarar las variables de coordenadas como privadas en `Cuadrilatero`.

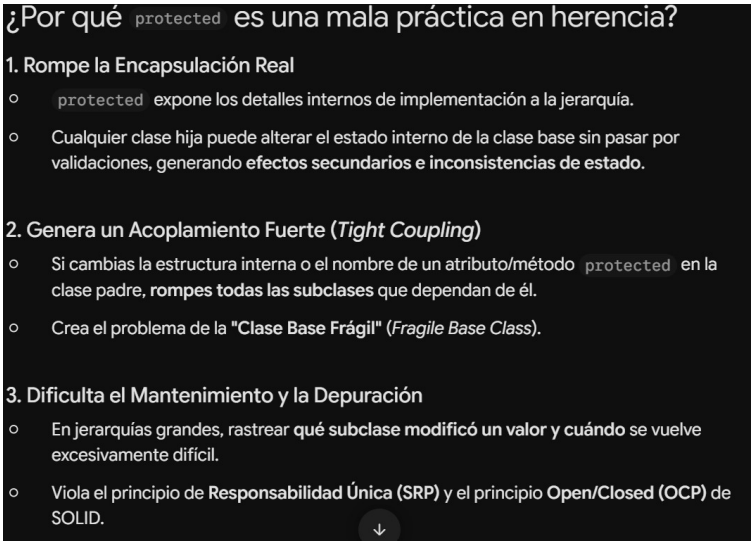


Figura 3.6: Consulta de investigación 6: Alex (Principios SOLID)

3.2. Diseño de clases y jerarquía de herencia

La clase `Punto` modela la unidad básica de posicionamiento cartesiano con coordenadas de tipo punto flotante de doble precisión (`double`), ofreciendo constructores sobrecargados y métodos de acceso y modificación con encapsulamiento estricto. La estructura UML de `Punto` se ilustra en la figura 3.7, mientras que el flujo de asignación en su constructor se detalla en la figura 3.8.

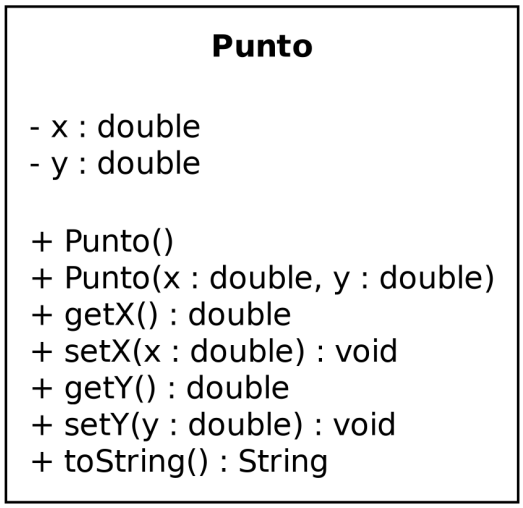


Figura 3.7: Diagrama UML de la clase Punto

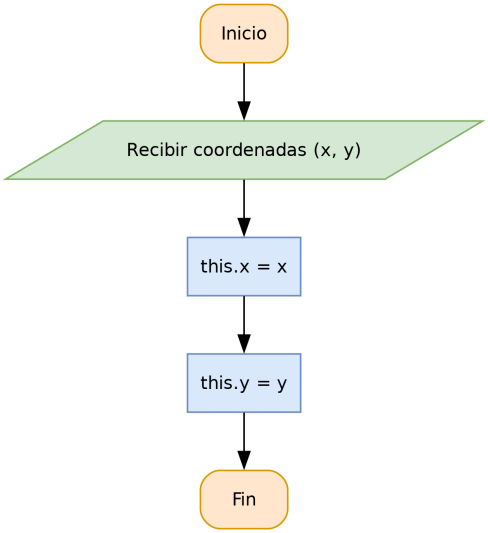


Figura 3.8: Diagrama de flujo de inicialización en la clase Punto

La superclase raíz `Cuadrilatero` contiene cuatro referencias privadas a instancias de `Punto`, identificadas como los vértices P_1, P_2, P_3 y P_4. Dicha clase provee únicamente métodos consultores y modificadores para cada vértice, así como una representación descriptiva en formato de texto. El diagrama estructural de `Cuadrilatero` se presenta en la figura 3.9, y su secuencia de asignación constructora se muestra en la figura 3.10.

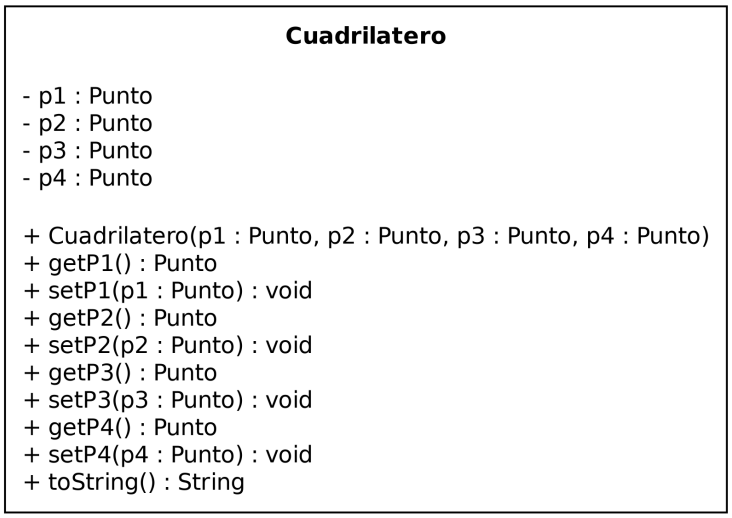


Figura 3.9: Diagrama UML de la clase Cuadrilatero

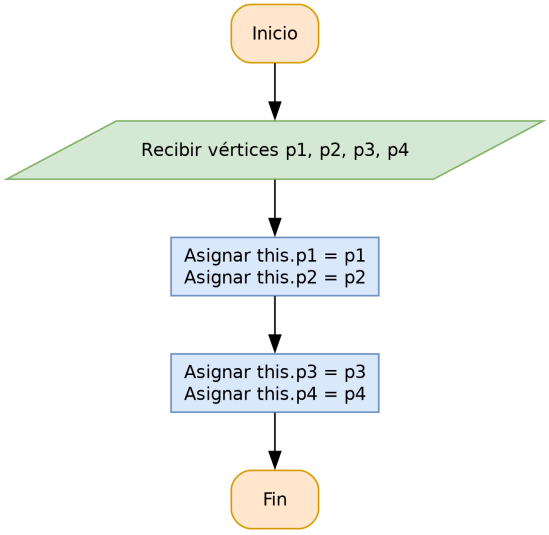


Figura 3.10: Diagrama de flujo de construcción en Cuadrilatero

La clase `Trapezoide` hereda directamente de `Cuadrilatero` y representa el primer nivel de especialización donde los lados inferior y superior son paralelos y horizontales. Implementa métodos para determinar la base mayor ($B = |x_2 - x_1|$), la base menor ($b = |x_3 - x_4|$) y la altura vertical ($h = |y_4 - y_1|$), definiendo el método `calcularArea()` con la fórmula analítica $A = \frac{(B+b)h}{2}$. El diagrama UML correspondiente aparece en la figura 3.11, y su dinámica de cálculo se expone en la figura 3.12.

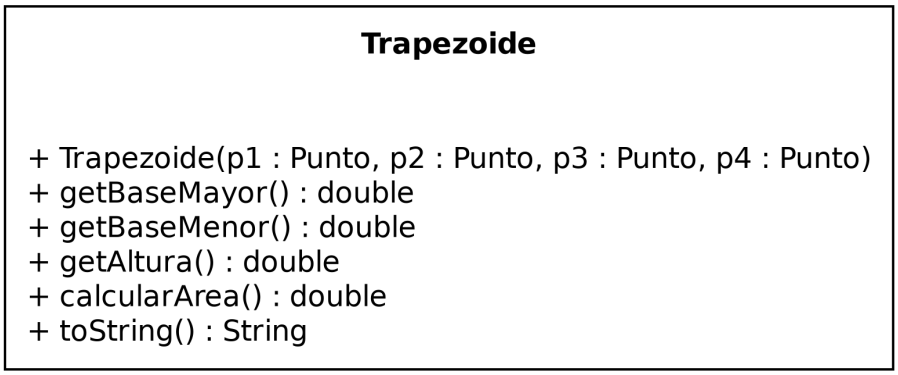


Figura 3.11: Diagrama UML de la clase Trapezoide

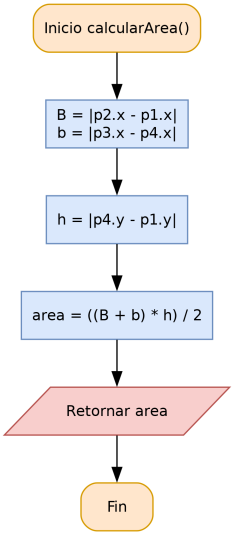


Figura 3.12: Diagrama de flujo del método calcularArea en Trapezoide

La clase `Paralelogramo` se deriva formalmente de `Trapezoide`. Dado que en un paralelogramo la base mayor y la menor son exactamente iguales ($B = b$), esta clase introduce el método consultor `getBase()` y sobrescribe `calcularArea()` para ejecutar directamente el producto $A = b \cdot h$. En la figura 3.13 se documenta el esquema UML de la clase, y en la figura 3.14 se visualiza la operación algorítmica de obtención del área.

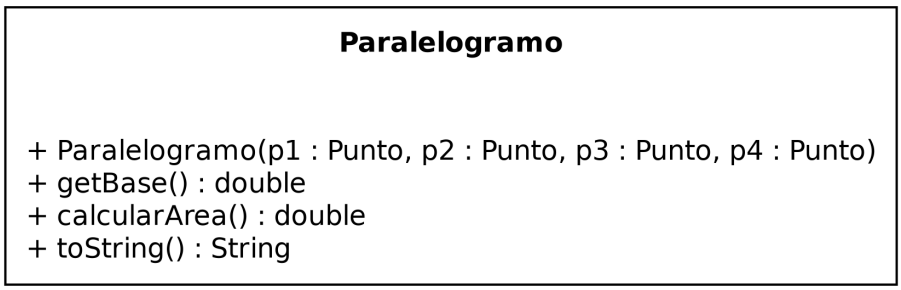


Figura 3.13: Diagrama UML de la clase Paralelogramo

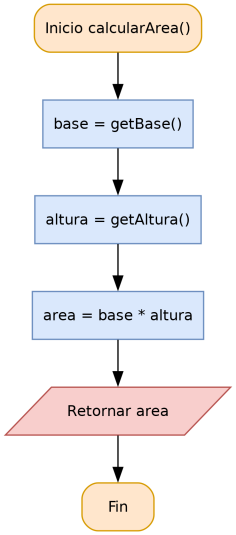


Figura 3.14: Diagrama de flujo del cálculo de área en Paralelogramo

La clase `Rectangulo` extiende a `Paralelogramo`, incorporando la restricción geométrica de ortogonalidad total en sus cuatro vértices. Mapea la base y la altura heredadas hacia métodos con semántica de dominio específica denominados `getLongitud()` y `getAncho()`, efectuando el cálculo del área mediante el producto directo de ambas dimensiones. La estructura de clases se aprecia en la figura 3.15, y el flujo operacional se presenta en la figura 3.16.

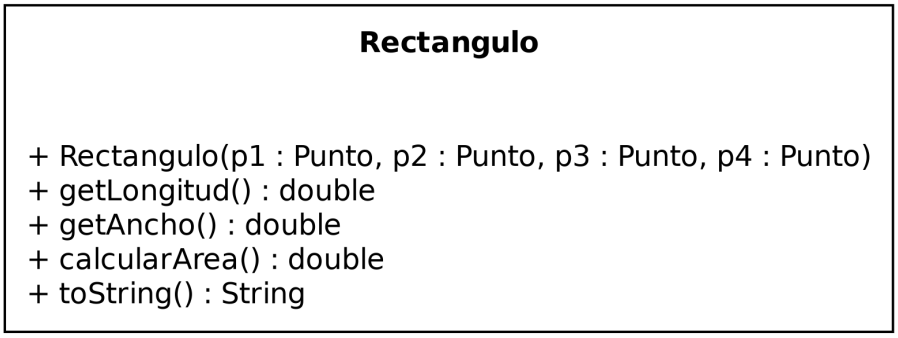


Figura 3.15: Diagrama UML de la clase Rectangulo

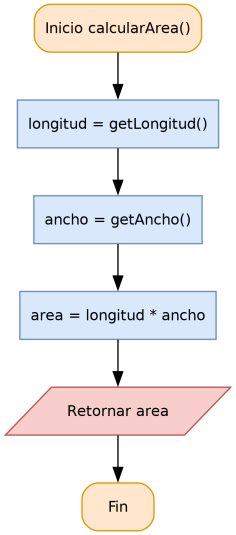


Figura 3.16: Diagrama de flujo de evaluación de área en Rectangulo

En la cima de la jerarquía profunda se localiza la clase `Cuadrado`, la cual hereda de `Rectangulo`. Al imponer que longitud y ancho posean exactamente la misma magnitud, define el método `getLado()` y sobrescribe `calcularArea()` aplicando la relación cuadrática $A = b^2 = \text{lado}^2$. El diagrama UML de `Cuadrado` se encuentra en la figura 3.17, y su flujo de ejecución se sintetiza en la figura 3.18.

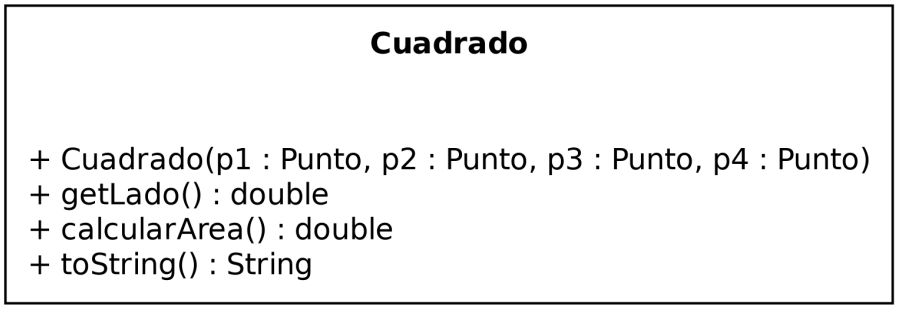


Figura 3.17: Diagrama UML de la clase Cuadrado

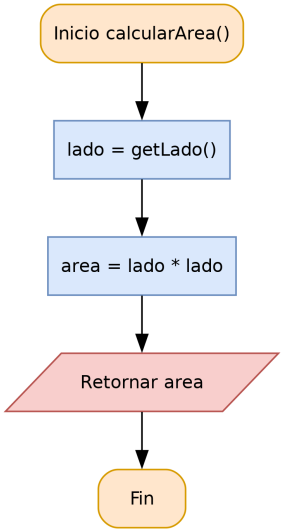


Figura 3.18: Diagrama de flujo del método calcularArea en Cuadrado

Finalmente, la clase `Main` funge como módulo conductor y de prueba. Instancia secuencialmente un objeto de cada figura geométrica con las coordenadas provistas en las especificaciones oficiales, reporta sus propiedades y despliega las áreas calculadas, omitiendo deliberadamente la operación de área para la superclase Cuadrilátero. El modelo de la clase principal se observa en la figura 3.19, y su algoritmo de despacho secuencial se describe en la figura 3.20.

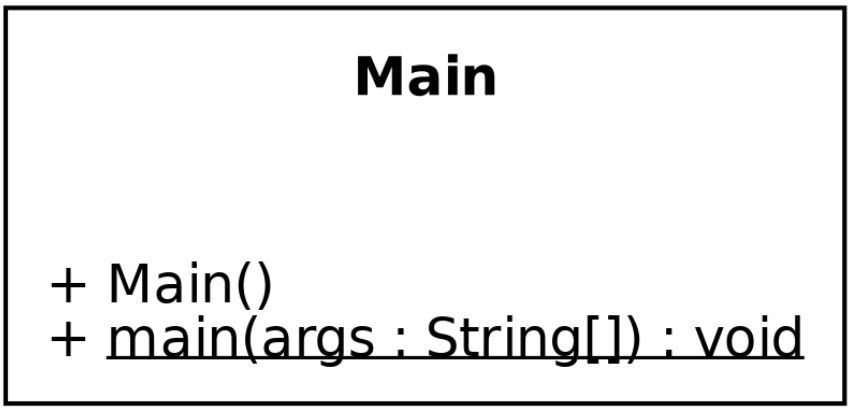


Figura 3.19: Diagrama UML de la clase controladora Main

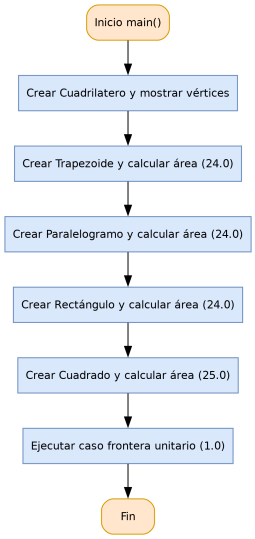


Figura 3.20: Diagrama de flujo de control secuencial en Main

4. Resultados

Para validar de forma fehaciente el comportamiento de las clases implementadas, se sometió el sistema a los casos de prueba estandarizados descritos en las especificaciones de la práctica, añadiendo además una verificación frontera de un cuadrado unitario. En el cuadro 1 se resume la correspondencia exacta entre las coordenadas suministradas a cada constructor, el valor de área calculado analíticamente de forma previa a la ejecución, y el resultado observado tras la corrida computacional en la máquina virtual de Java.

Tabla 1: Comparativa de casos de prueba entre esperado calculado y observado ejecutado.

Figura	**Vértices de entrada** (x, y)	**Esperado**	**Observado**	**Diferencia**	**Estado**
Cuadrilátero	$P_1(1,1), P_2(5,1), P_3(4,4), P_4(2,3)$	N/A	N/A	0.00	Válido
Trapezoide	$P_1(0,0), P_2(8,0), P_3(6,4), P_4(2,4)$	24.00	24.00	0.00	Exitoso
Paralelogramo	$P_1(0,0), P_2(6,0), P_3(8,4), P_4(2,4)$	24.00	24.00	0.00	Exitoso
Rectángulo	$P_1(0,0), P_2(6,0), P_3(6,4), P_4(0,4)$	24.00	24.00	0.00	Exitoso
Cuadrado	$P_1(0,0), P_2(5,0), P_3(5,5), P_4(0,5)$	25.00	25.00	0.00	Exitoso
Cuadrado frontera	$P_1(0,0), P_2(1,0), P_3(1,1), P_4(0,1)$	1.00	1.00	0.00	Exitoso

Los resultados confirman que el modelo analítico y el modelo computacional coinciden con absoluta precisión numérica. En la figura 4.1 se documenta el proceso de compilación limpia de los archivos fuentes hacia la carpeta de binarios mediante la herramienta Freeze.

```
Compilación exitosa: 7 clases compiladas limpiamente en test/.
```

Figura 4.1: Evidencia de compilación limpia con el compilador javac

Para verificar la fidelidad de las salidas en tiempo de ejecución, se llevaron a cabo corridas tanto a nivel de clases compiladas individuales como desde el archivo empaquetado. En la figura 4.2 se aprecia la salida por consola obtenida al ejecutar el programa directamente desde el directorio de clases compiladas. Por su parte, en la figura 4.3 se valida la ejecución autónoma del trabajo completo empaquetado en el archivo JAR distribuible, constatando la equivalencia determinista de resultados en ambas modalidades de despliegue.



Figura 4.2: Salida por consola de la ejecución directa desde el directorio test



Figura 4.3: Ejecución autónoma del programa desde el paquete distribuible JAR

5. Conclusiones

El diseño e implementación de la jerarquía profunda para la familia de los cuadriláteros confirma la solidez conceptual de la programación orientada a objetos para trasladar relaciones taxonómicas y geométricas complejas a estructuras de software estructuradas y mantenibles. Al situar a la clase Cuadrilátero como superclase contenedora de las coordenadas básicas e incorporar subclases descendientes en una cadena lineal continua (Trapezoide, Paralelogramo, Rectángulo y Cuadrado), se logró un nivel sobresaliente de reutilización de código, permitiendo que las clases derivadas exploten directamente los atributos espaciales heredados sin necesidad de redefinirlos en cada estrato.

Asimismo, la exclusión deliberada de un método de área en la superclase Cuadrilátero y su introducción formal a partir de la clase Trapezoide demuestra una aplicación rigurosa de los principios de encapsulamiento y coherencia polimórfica. Cada nivel de la jerarquía reflejó fielmente la fórmula matemática que gobierna su geometría particular, simplificando la evaluación de la superficie conforme las restricciones de paralelismo, ortogonalidad y congruencia se intensificaron. La coincidencia exacta entre los valores esperados analíticamente y las salidas generadas tanto en ejecución directa como desde el archivo empaquetado JAR valida la corrección técnica de la solución entregada.

Referencias

[1] P. Deitel y H. Deitel, *Java How to Program (Early Objects)*, 11.ª ed. Boston, EE. UU.: Pearson, 2018.

[2] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, y D. Smith, *The Java Language Specification (Java SE 21 Edition)*. Santa Clara, EE. UU.: Oracle Corporation, 2023.

[3] J. Bloch, *Effective Java*, 3.ª ed. Boston, EE. UU.: Addison-Wesley Professional, 2018.